Arbitrary Precision Arithmetic Library in Java

Your Name

May 2, 2025

Abstract

This project implements a Java-based arbitrary-precision arithmetic library that can handle extremely large integers and floating-point numbers. It surpasses the limitations of built-in types like `int` and `double`. The library is built around two core classes: `AInteger` for integer operations and `AFloat` for floating-point calculations. It supports addition, subtraction, multiplication, division, and modulus (in integers).

1 Introduction

The goal of this project was to develop an arbitrary-precision arithmetic library that supports basic arithmetic operations on very large integers and floating-point numbers. Java's built-in data types like `int` and `double` are limited in precision and range. This library aims to overcome these limitations by implementing classes that handle large numbers and perform arithmetic operations on them.

Structure of Project

- Java Project
  - arbitraryarithmetic
    - AFloat.java
    - AInteger.java
    - commonMethod.java
    - aarithmetic.jar
  - MyInfArith.java
  - build.xml
  - build
    - arbitraryarithmetic
      - AFloat.class
      - AInteger.java

        - commonMethod.java
    * MyInfArith.class
- Script.py
- compile.py
- default-test-case.txt
- dockerfile
- Latex documentation
- README.md

2 Overview of the Library

The library includes two main classes:

- **AInteger**: A class that supports arbitrary-precision integer arithmetic.
- **AFloat**: A class that supports arbitrary-precision floating-point arithmetic.

Both classes support operations such as addition, subtraction, multiplication, division, and modulus. These classes implement basic algorithms for handling large numbers using string manipulation to store and operate on digits.

3 Class Implementations

3.1 AInteger Class

The `AInteger` class implements arbitrary-precision integer arithmetic. Key operations include:

- **Addition**: Handles both positive and negative numbers, ensuring the result is correctly signed and removing any leading zeros.
- **Subtraction**: Supports subtraction of both positive and negative numbers and ensures correct sign handling.
- **Multiplication**: Uses grade-school multiplication, breaking down the multiplication of each digit, and correctly appending zeros for positional value.
- **Division**: Implements long division to handle division of large integers. Special handling for division by zero is included.
- **Modulus**: Uses the division result to calculate the remainder.

3.2 AFloat Class

The `AFloat` class is designed to represent floating-point numbers using arbitrary precision. It separates the number into two parts: an integer part and a fractional part, both of which are stored as `AInteger` objects.

The constructor handles input parsing by identifying the decimal point and splitting the number accordingly. Both parts are then converted to `AInteger` instances. Sign handling is inherited from the input, and trailing/leading zeros are trimmed as needed.

Key operations include:

- **Addition and Subtraction**: Before performing arithmetic, the fractional parts are padded with zeros to equalize their lengths. The integer and fractional parts are then added or subtracted separately using `AInteger`. After the operation, the parts are normalized to restore the decimal format.
- **Multiplication**: Both numbers are converted to equivalent integers by removing their decimal points. The integer multiplication result is then adjusted by reinserting the decimal at the correct location based on the combined number of decimal places in the operands.
- **Division**: The division logic scales both operands to eliminate decimals, performs integer division, and then formats the result to reintroduce the decimal with appropriate rounding and precision. Division by zero is detected and handled with appropriate error messages.

The class ensures precision and correctness by relying on the `AInteger` operations, effectively simulating floating-point arithmetic with full control over scale and accuracy.

4 Usage of the Library

4.1 Command-Line Interface

The project includes a command-line interface (CLI) through the `MyInfArith.java` class, which parses user input and invokes the corresponding methods in `AInteger` or `AFloat`.

Usage pattern:

```
java MyInfArith <int/float> <add/sub/mul/div> <num1> <num2>
```

Examples:

```
java MyInfArith int add 236500782249129494973109332402504293978326246711379838638440149
java MyInfArith float div 3227 555
java MyInfArith float add 5.25 0.75
```

The program uses the `compile.py` Python script to automate the compilation (via Ant) and execution of test cases. Test cases are stored in a file and can be executed repeatedly or extended interactively. The CLI handles input validation, fallback to default cases, and dynamic storage of new cases.

4.2 Test Framework and Case Handling

The test framework is driven by a Python script that compiles the Java code using Apache Ant (via `compile.py`) and provides an interactive command-line interface. Upon running, the user is prompted to input a test case in the format:

```
<int/float> <add/sub/mul/div> <num1> <num2>
```

For example:

```
Enter the input (or leave it empty to run default test cases):
int add 2 3
5
Do you want to add this to the default test cases? (y/n)
y
Test case saved to default_test_case.txt.
Do you want to test another test case? (y/n):
n
```

If the input is left empty, all saved test cases from the default file will be executed. For each new test case, the user is prompted whether to save it to the default file and whether they wish to continue testing additional cases. This setup allows dynamic expansion of the test suite and provides a simple interface for repeatable testing without modifying code or manually rerunning commands.

5 Conclusion

This arbitrary-precision arithmetic library provides a robust solution for performing large number arithmetic operations in Java. By using string manipulation to handle large numbers, the library can support integers and floating-point numbers of arbitrary size. The AFloat class, in particular, demonstrates a thoughtful approach to high-precision decimal handling by leveraging internal integer arithmetic. This project demonstrates the power of custom data structures for handling specific numerical tasks that go beyond built-in data types.